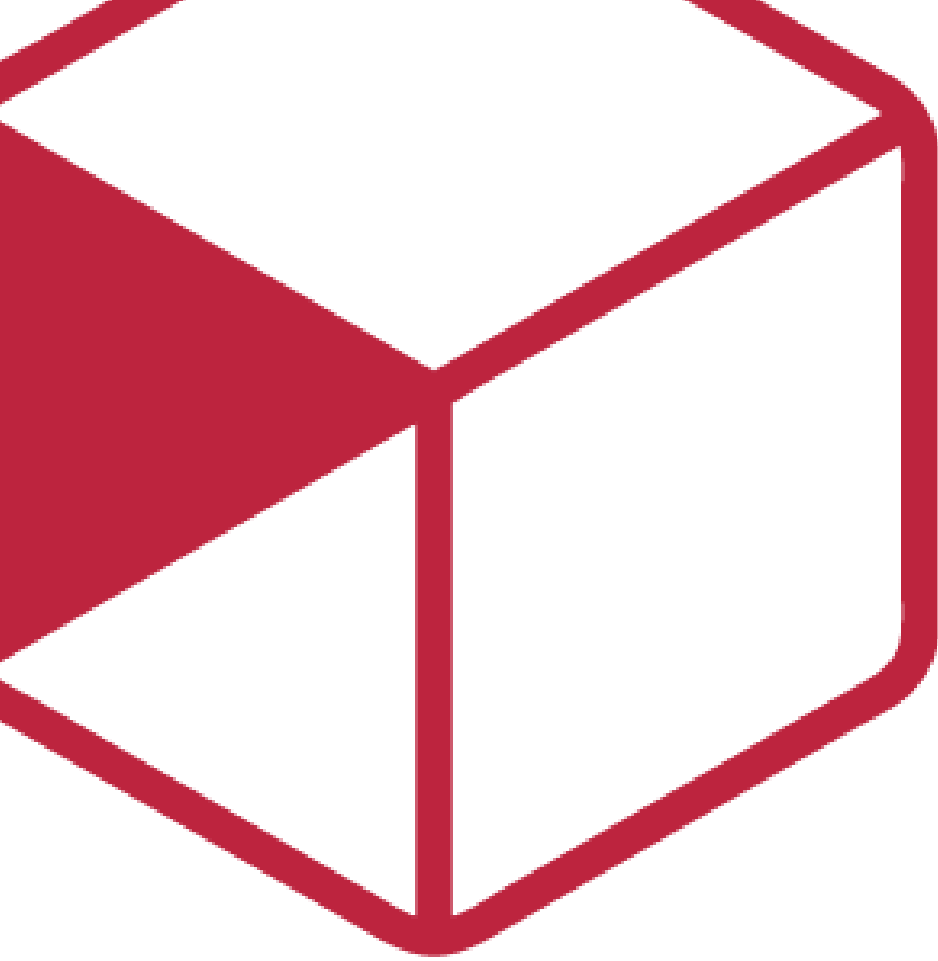




Reglas de negocio y especificación funcional

Diseño de Solución — Backend

FORMATO	N/A
PROYECTO	PI-DES-26 Entrega del Soporte Operativo a SaP y CRFS
REFERENCIA	N/A
VERSIÓN	1.0
FECHA	19 jun 2026
AUTOR	Jose Alberto Perez Pantoja
REVISOR	Juan David García Cruz

Contenido

Requisito R16A-RE-FU-001 — Catálogo de Cuentas Bancarias del grupo PROQUIFA	3
Importante	3
Introducción	3
1. Propósito del documento	3
2. Alcance	3
Específicamente incluye	3
No se consideran	4
Guías de Resolución (TSG) requeridas	4
Visión general del diseño	4
1. Objetivo técnico	4
2. Componentes involucrados	4
Diseño funcional detallado	5
1. Flujo — Consulta del catálogo bancario por módulo consumidor	5
2. Flujo — Consulta de una cuenta por Id	6
3. Flujo — Baja lógica (operación de Soporte a la Producción)	6
4. Criterios de aceptación — cobertura técnica	6
5. Reglas técnicas aplicadas	7
Diseño de componentes	7
1. Modelo de datos	7
1.1 EmpresaDatosBancarios — Tabla principal (cambio R16)	7
1.2 vEmpresaDatosBancarios — Vista nueva R16	8
2. Flujo de actualización del modelo EF (EDMX)	9
3. Corrección ObtenerTodos() — GAP-03	10
4. Nuevo vEmpresaDatosBancariosBO	10
5. Nuevo vEmpresaDatosBancariosController	11
6. Diagrama de secuencia	12
Análisis de compatibilidad: CLABE (MEX) vs CCI (PER)	12
1. Cobertura por campo	13
2. Cambios R16 planeados — impacto sobre CCI	13
3. Punto crítico: nombre del campo Clabe	13
4. Requisitos pendientes para habilitar CCI (Perú — fuera de alcance R16)	14
5. Conclusión	14
Impacto técnico	14
1. Orden de ejecución y archivos	14
2. Archivos sin cambio en R16	15
Manejo de errores y excepciones	15
Estrategia de pruebas	16
1. Pruebas funcionales (Criterios de Aceptación en DEV)	16
2. Pruebas técnicas	16



a. Unitarias	16
b. Integración	16
3. Casos críticos	16
Control de versiones	17



Requisito R16A-RE-FU-001 — Catálogo de Cuentas Bancarias del grupo PROQUIFA

FORMATO	Arquitectura
PROYECTO	R16 - Adquisiciones
REFERENCIA	AUI-FOR-01
VERSIÓN	1.0
FECHA	19 jun 2026
AUTOR	Jose Alberto Perez Pantoja
REVISOR	Juan David García Cruz

Importante

Antes de iniciar el desarrollo, verificar:

- ¿El programador sabe qué flujo implementar?
- ¿Sabe qué pasa si algo falla?
- ¿Sabe qué reglas no puede romper?
- ¿Sabe qué pruebas debe pasar?
- ¿Sabe dónde impacta?

Este documento sigue el estándar IEEE 1016 "Software Design Description" y cubre únicamente el diseño del backend para este requisito.

Introducción

1. Propósito del documento

Definir el diseño de la solución técnica backend para el requisito **R16A-RE-FU-001 (Catálogo de Cuentas Bancarias del grupo PROQUIFA)**, describiendo los cambios de BD, el flujo de actualización del modelo EF, los componentes nuevos y las reglas técnicas de implementación.

Nota: *Este documento cubre exclusivamente el Back-end. No redefine requisitos funcionales ni cubre Front-end.*

2. Alcance

Específicamente incluye

- Script ALTER TABLE: agregar columna IdRegion a EmpresaDatosBancarios (BD ProquifaDotNet).
- Script CREATE VIEW: crear vista vEmpresaDatosBancarios con JOIN completo.

- Actualización del modelo EF6 (EDMX) en Core.Pqf tras cada cambio en BD.
- Generación del NuGet de Core.Pqf e instalación en ProquifaDotNet-R17.
- Corrección de ObtenerTodos() en EmpresaDatosBancariosB0.Extensions.cs.
- Nuevo BO: vEmpresaDatosBancariosB0 en Logic.Pqf.Catalogos.
- Nuevo Controller: vEmpresaDatosBancariosController en WebApi.Catalogos.

No se consideran

- Interfaz gráfica de usuario (UI): no se desarrolla en R16 para este catálogo.
- Endpoints de alta, baja o modificación: gestión exclusiva de Soporte a la Producción mediante **Guías de Resolución (TSG)** — no se exponen endpoints para estas operaciones.
- Cuentas Perú (Golocaer S.A.C.): modelo SUNAT no definido — fuera de alcance R16.

Guías de Resolución (TSG) requeridas

Las siguientes operaciones de gestión del catálogo se documentan como Guías de Resolución (TSG) para ejecución exclusiva por Soporte a la Producción:

TSG	Operación	Descripción
TSG-FU-001-01	Alta de cuenta bancaria	INSERT en DatosBancarios + EmpresaDatosBancarios con IdRegion
TSG-FU-001-02	Baja lógica de cuenta	UPDATE Activo = 0 — nunca DELETE físico
TSG-FU-001-03	Modificación de datos bancarios	UPDATE sobre DatosBancarios con actualización de FechaUltimaActualizacion

Visión general del diseño

1. Objetivo técnico

Exponer el Catálogo de Cuentas Bancarias del grupo PROQUIFA como fuente única de consulta para todos los módulos consumidores del sistema, regionalizado por el usuario autenticado (MEX / PER), garantizando que solo se ofrezcan cuentas vigentes (Activo = 1) y que los registros dados de baja se conserven para trazabilidad histórica (sin DELETE físico).

2. Componentes involucrados

Aplicativo	Componente	Responsabilidad	Ubicación
BD ProquifaDotNet	EmpresaDatosBancarios	Tabla principal del catálogo — agrega `IdRegion` en R16	BD

BD ProquifaDotNet	vEmpresaDatosBancarios	Nueva vista R16 — JOIN completo: empresa + banco + moneda + región	BD
Core.Pqf	EDMX (ProquifaDotNetModel)	Modelo EF6 — se actualiza tras cada cambio en BD; genera NuGet	ProquifaDotNetContext\
Logic.Pqf.Catalogos	EmpresaDatosBancariosB0.Extensions	Modificar — ObtenerTodos() agrega filtro Activo = 1	Empresas\DatosBancarios\
Logic.Pqf.Catalogos	vEmpresaDatosBancariosB0	Nuevo — consulta paginada y por Id sobre la vista	Empresas\DatosBancarios\
WebApi.Catalogos	vEmpresaDatosBancariosController	Nuevo — expone GET y POST del catálogo bancario regionalizado	Controllers\Configuracion\Empresas\

Diseño funcional detallado

1. Flujo — Consulta del catálogo bancario por módulo consumidor

1. El módulo consumidor (ej. Proforma, Validar Cobro) necesita listar cuentas bancarias del grupo.
2. El módulo llama al endpoint:

```
POST /vEmpresaDatosBancarios
Body: QueryInfo { skip, take, filtros, idRegion? }
```

3. vEmpresaDatosBancariosController recibe la petición.
4. El Controller determina el IdRegion a usar:
5. Si info.IdRegion tiene valor → usa ese (validando que el usuario tenga permiso para consultar esa región)
6. Si no → extrae IdRegion del +on

```
db.vEmpresaDatosBancarios
    .Where(v => v.IdRegion == idRegion && v.Activo)
    .OrderBy(v => v.EmpresaPrefijo)
    .Skip(info.Skip).Take(info.Take)
```



7. El Controller llama a `vEmpresaDatosBancariosBO.Lista(info, idRegion)`.
8. El BO consulta `ProquifaDotNetEntities`:
9. EF traduce la consulta a SQL contra `vEmpresaDatosBancarios` en BD.
10. El BO retorna `QueryResult<vEmpresaDatosBancarios>` con Total y Results.
11. El Controller responde HTTP 200 con el resultado paginado.
12. El módulo consumidor renderiza las cuentas sin mantener copia local.

2. Flujo — Consulta de una cuenta por Id

El módulo llama:

GET `/vEmpresaDatosBancarios?idEmpresaDatosBancarios={guid}`

El Controller llama a `vEmpresaDatosBancariosBO.ObtenerPorId(id)`.

El BO filtra `Activo = true` — retorna null si no existe o está inactiva.

Controller responde 200 OK con la cuenta o 204 NoContent.

3. Flujo — Baja lógica (operación de Soporte a la Producción)

*Este flujo NO se expone como endpoint. La baja la ejecuta Soporte a la Producción directamente en BD mediante la **Guía de Resolución correspondiente (TSG)**.*

```
UPDATE dbo.EmpresaDatosBancarios
SET Activo = 0,
    FechaUltimaActualizacion = GETDATE()
WHERE IdEmpresaDatosBancarios = @IdCuenta;
```

El registro permanece en BD con `Activo = 0`. Los módulos dejan de recibirlo automáticamente al filtrar `Activo = 1`.

#	Método HTTP	Ruta	Parametros	Salida	Acción
Catalogos	POST	<code>/vEmpresaDatosBancarios</code>	<pre>QueryInfo { IdRegion?: Guid }</pre>	<pre>QueryResult { Total: int, Results: [{ IdEmpresaDatosBancarios: Guid, EmpresaPrefijo: string, Banco: string, NumeroDeCuenta: string, Clabe: string, ClaveMoneda: string, ... }] }</pre>	Lista paginada de cuentas bancarias filtrada por región

Catálogos	GET	/vEmpresa DatosBancarios	?idEmpresaDatosBancarios: Guid	vEmpresaDatosBancarios { IdEmpresaDatosBancarios: Guid, EmpresaPrefijo: string, Banco: string, NumeroDeCuenta: string, Clabe: string ClaveMoneda: string, ... }	Obtener cuenta bancaria por Id
-----------	-----	-----------------------------	-----------------------------------	---	---

4. Criterios de aceptación — cobertura técnica

CA	Descripción	Estado	Justificación
A1	Módulo obtiene cuentas vigentes al consultar el catálogo	Cubierto	Flujo 1: WHERE Activo = 1 en BO sobre la vista
A2	Filtrado por empresa emisora	Cubierto	IdRegion del token filtra cuentas por región del usuario
B1	Solo cuentas vigentes se ofrecen en módulos operativos	Cubierto	BO aplica Activo = true — nunca expone inactivas
B2	Cuentas dadas de baja conservan registro para trazabilidad	Cubierto	UPDATE Activo=0, nunca DELETE; registro persiste en BD
C1	No existe pantalla de gestión del catálogo en la aplicación	Cubierto	Controller no expone PUT ni DELETE
C2	La baja se realiza a nivel lógico por Soporte a la Producción	Cubierto	Flujo 3 documentado para BD — operación guiada por TSG

5. Reglas técnicas aplicadas

Regla	Descripción
-------	-------------



RT-01	Prohibido <code>DELETE</code> físico sobre <code>EmpresaDatosBancarios</code> . Toda baja es <code>UPDATE Activo = 0</code> .
RT-02	Filtro <code>Activo = true</code> aplicado siempre en el BO. No se delega al módulo caller.
RT-03	<code>IdRegion</code> se extrae del token — nunca se acepta como parámetro externo en ruta ni body.
RT-04	Los módulos consumen <code>vEmpresaDatosBancarios</code> (vista). Prohibido consultar la tabla directamente.
RT-05	Sin catálogos paralelos ni copias locales de cuentas en otros módulos.
RT-06	Cuentas PE (GOLPERU) fuera de alcance R16 — no se incluyen hasta definir modelo SUNAT.
RT-07	Toda modificación al EDMX requiere generar nuevo NuGet de <code>Core.Pqf</code> y actualizar referencia en <code>ProquifaDotNet-R17</code> .

Diseño de componentes

1. Modelo de datos

1.1 `EmpresaDatosBancarios` — Tabla principal (cambio R16)

Columna	Tipo	Nullable	Descripción
<code>IdEmpresaDatosBancarios</code>	<code>uniqueidentifier</code>	NO	PK
<code>IdDatosBancarios</code>	<code>uniqueidentifier</code>	NO	FK → <code>DatosBancarios</code>
<code>IdEmpresa</code>	<code>uniqueidentifier</code>	SÍ	FK → Empresa
<code>FechaRegistro</code>	<code>datetime</code>	NO	Fecha de alta
<code>FechaUltimaActualizacion</code>	<code>datetime</code>	NO	Fecha de modificación
<code>Activo</code>	<code>bit</code>	NO	1 = vigente; 0 = baja lógica

 `IdRegion`	`uniqueidentifier`	SÍ	NUEVO R16 — FK → `Region` (MEX / PER)
--	--------------------	----	---

Script Paso 1:

```
-- 1.1 Agregar columna IdRegion
ALTER TABLE dbo.EmpresaDatosBancarios
    ADD IdRegion uniqueidentifier NULL
    CONSTRAINT FK_EmpresaDatosBancarios_Region
    FOREIGN KEY REFERENCES dbo.Region(IdRegion);

-- 1.2 Poblar con región México para registros existentes
DECLARE @IdMexico uniqueidentifier = '60390fda-7773-4ba1-8120-cb874f3a3a53';

UPDATE dbo.EmpresaDatosBancarios
SET     IdRegion                = @IdMexico,
        FechaUltimaActualizacion = GETDATE()
WHERE  IdRegion IS NULL;
```

 **Ejecutar antes del Paso 2 (la vista depende de esta columna).**

1.2 vEmpresaDatosBancarios — Vista nueva R16

Prerrequisito: Paso 1 completado.

Columnas principales:

Columna	Origen	Descripción
IdEmpresaDatosBancarios	EmpresaDatosBancarios	PK del registro
EmpresaPrefijo	Empresa.Prefijo	GOL / MUN / PRO / PQF
EmpresaAlias	Empresa.Alias	Nombre corto
IdRegion	EmpresaDatosBancarios	FK — Región
RegionClave	Region.ClaveISO	MEX / PER
Banco	catBanco.Banco	Institución bancaria
NumeroDeCuenta	DatosBancarios	Número de cuenta
Clabe	DatosBancarios	CLABE 18 dígitos
ClaveMoneda	catMoneda.ClaveMoneda	MXN / USD
Activo	EmpresaDatosBancarios	1 = vigente

Script Paso 2:

```
CREATE OR ALTER VIEW dbo.vEmpresaDatosBancarios
AS
SELECT
    edb.IdEmpresaDatosBancarios,
    edb.IdEmpresa,
    e.Prefijo      AS EmpresaPrefijo,
    e.Alias       AS EmpresaAlias,
```

```

    edb.IdRegion,
    r.Nombre          AS Region,
    r.ClaveISO        AS RegionClave,
    edb.IdDatosBancarios,
    db.IdCatBanco,
    b.Banco,
    b.Clave           AS ClaveBanco,
    db.NumeroDeCuenta,
    db.Clabe,
    db.Beneficiario,
    db.Sucursal,
    db.IdCatMoneda,
    m.ClaveMoneda,
    m.Moneda,
    edb.FechaRegistro,
    edb.FechaUltimaActualizacion,
    edb.Activo
FROM      dbo.EmpresaDatosBancarios edb
LEFT JOIN dbo.Empresa                e ON edb.IdEmpresa    = e.IdEmpresa
LEFT JOIN dbo.Region                r ON edb.IdRegion      = r.IdRegion
INNER JOIN dbo.DatosBancarios       db ON edb.IdDatosBancarios = db.IdDatosBancarios
LEFT JOIN dbo.catBanco              b ON db.IdCatBanco     = b.IdCatBanco
LEFT JOIN dbo.catMoneda             m ON db.IdCatMoneda    = m.IdCatMoneda;

```

2. Flujo de actualización del modelo EF (EDMX)

Paso 1 – ALTER TABLE

↓
 Actualizar EDMX en Core.Pqf (Update Model from Database)
 → regenera EmpresaDatosBancarios.cs con IdRegion

↓
 Paso 2 – CREATE VIEW

↓
 Actualizar EDMX de nuevo
 → agrega vEmpresaDatosBancarios.cs al contexto

↓
 Generar nuevo NuGet de Core.Pqf

↓
 Actualizar NuGet en ProquifaDotNet-R17

↓
 Crear vEmpresaDatosBancariosB0 + vEmpresaDatosBancariosController

3. Corrección obtenerTodos() — GAP-03

Archivo:

```

Logic.Pqf.Catalogos\Empresas\DatosBancarios\EmpresaDatosBancariosB0.Extensions.cs
// ANTES – devuelve cuentas no vigentes (bug)
public List<EmpresaDatosBancarios> ObtenerTodos()
{
    using (var db = new ProquifaDotNetEntities())
    {
        return db.EmpresaDatosBancarios.ToList();
    }
}

```



```
    }  
}  
  
// DESPUÉS - solo cuentas vigentes  
public List<EmpresaDatosBancarios> ObtenerTodos()  
{  
    using (var db = new ProquifaDotNetEntities())  
    {  
        return db.EmpresaDatosBancarios  
            .Where(x => x.Activo)  
            .ToList();  
    }  
}
```

4. Nuevo vEmpresaDatosBancariosB0

Archivo: Logic.Pqf.Catalogos\Empresas\DatosBancarios\vEmpresaDatosBancariosB0.cs

```
using Core.CrudTools.Optimization;  
using Core.Pqf.ProquifaDotNetContext;  
using System;  
using System.Linq;  
  
namespace Logic.Pqf.Catalogos.Empresas.DatosBancarios  
{  
    public class vEmpresaDatosBancariosB0  
    {  
        public vEmpresaDatosBancarios ObtenerPorId(Guid idEmpresaDatosBancarios)  
        {  
            using (var db = new ProquifaDotNetEntities())  
            {  
                return db.vEmpresaDatosBancarios  
                    .FirstOrDefault(v =>  
                        v.IdEmpresaDatosBancarios == idEmpresaDatosBancarios &&  
                        v.Activo);  
            }  
        }  
  
        public QueryResult<vEmpresaDatosBancarios> Lista(QueryInfo info, Guid idRegion)  
        {  
            using (var db = new ProquifaDotNetEntities())  
            {  
                var query = db.vEmpresaDatosBancarios  
                    .Where(v => v.IdRegion == idRegion && v.Activo)  
                    .AsQueryable();  
  
                var total = query.Count();  
                var items = query  
                    .OrderBy(v => v.EmpresaPrefijo)  
                    .Skip(info.Skip)  
                    .Take(info.Take)  
                    .ToList();  
            }  
        }  
    }  
}
```



```

        return new QueryResult<vEmpresaDatosBancarios>
        {
            Total    = total,
            Results  = items
        };
    }
}
}
}
}

```

5. Nuevo vEmpresaDatosBancariosController

Archivo:

WebApi.Catalogos\Controllers\Configuracion\Empresas\vEmpresaDatosBancariosController.cs

```

using System;
using System.Net.Http;
using System.Web.Http;
using System.Web.Http.Description;
using Core.CrudTools.Optimization;
using Core.Pqf.ProquifaDotNetContext;
using Core.Pqf.WebApi.ControllerOperations;
using Logic.Pqf.Catalogos.Empresas.DatosBancarios;

namespace WebApi.Controllers.Configuracion.Empresas
{
    public class vEmpresaDatosBancariosController : BaseApiController
    {
        // GET /vEmpresaDatosBancarios?idEmpresaDatosBancarios={guid}
        [HttpGet]
        [Route("vEmpresaDatosBancarios")]
        [ResponseType(typeof(vEmpresaDatosBancarios))]
        public HttpResponseMessage Obtener(Guid idEmpresaDatosBancarios)
        {
            return TryExecute(() =>
            {
                var bo      = new vEmpresaDatosBancariosBO();
                var result = bo.ObtenerPorId(idEmpresaDatosBancarios);
                return result != null
                    ? Request.CreateResponse(System.Net.HttpStatusCode.OK, result)
                    : Request.CreateResponse(System.Net.HttpStatusCode.NoContent);
            });
        }

        // POST /vEmpresaDatosBancarios - Body: QueryInfo
        [HttpPost]
        [Route("vEmpresaDatosBancarios")]
        [ResponseType(typeof(QueryResult<vEmpresaDatosBancarios>))]
        public HttpResponseMessage QueryResult([FromBody] QueryInfo info)
        {
            return TryExecute(() =>
            {

```

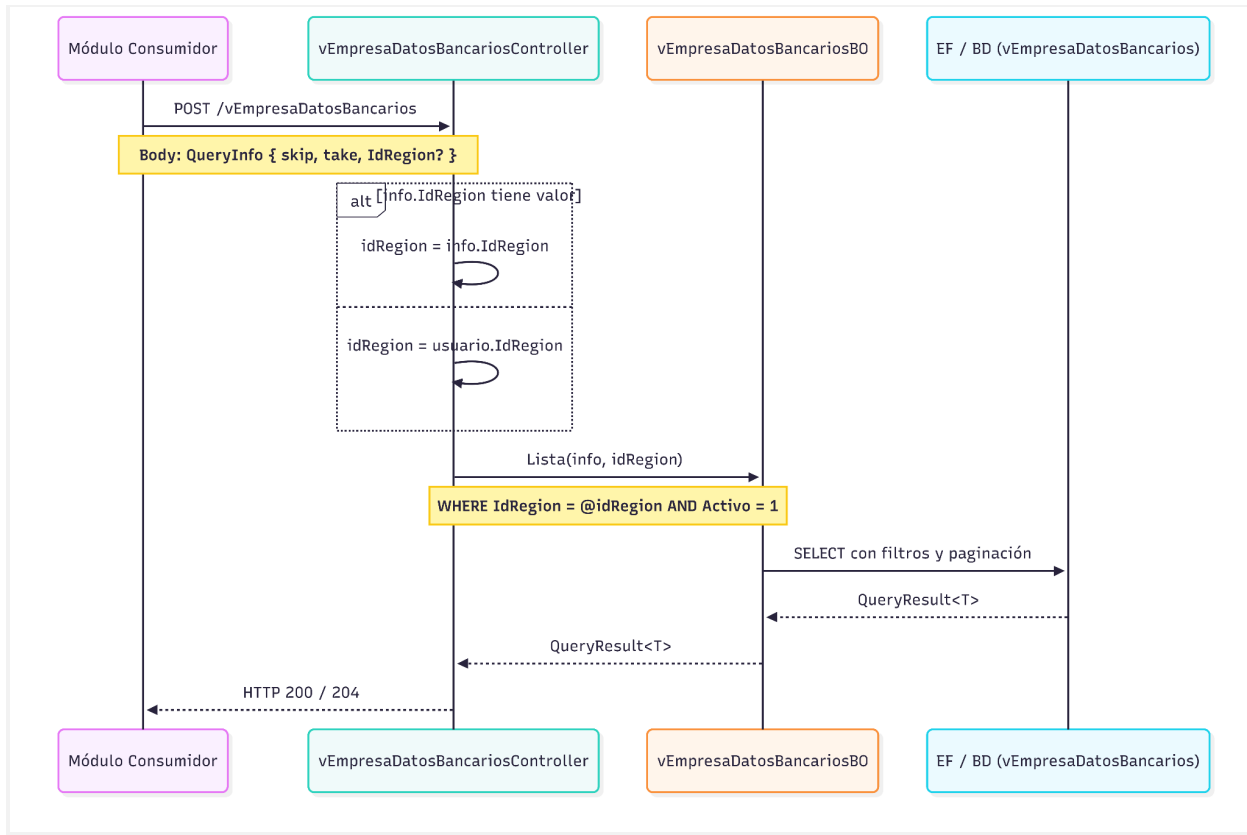


```

        var idRegion = ObtenerUsuarioAutenticado().IdRegion;
        var bo      = new vEmpresaDatosBancariosBO();
        var result  = bo.Lista(info, idRegion);
        return result != null
            ? Request.CreateResponse(System.Net.HttpStatusCode.OK, result)
            : Request.CreateResponse(System.Net.HttpStatusCode.NoContent);
    });
}
}
}

```

6. Diagrama de secuencia



Análisis de compatibilidad: CLABE (MEX) vs CCI (PER)

Análisis de si la estructura actual de DatosBancarios y los cambios R16 planeados soportan ambos estándares interbancarios.

1. Cobertura por campo



Campo en `DatosBancarios`	CLABE MEX (18 dígitos)	CCI PER (20 dígitos)	Veredicto
Clabe varchar(200)	✓ 18 dígitos caben	✓ 20 dígitos caben	Soporta ambos — nombre engañoso para PER (ver §3)
NumeroDeCuenta varchar(20)	✓ cuenta interna 11 dígitos	✓ número cuenta 11 dígitos	Cubierto
Sucursal varchar(50)	⚠ MEX usa código ciudad/plaza (3 dígitos) — no se almacena hoy	✓ código agencia PER (3 dígitos) — cabe aquí	Reutilizable; sin semántica formal
IdCatBanco → catBanco.Clave varchar(8)	✓ código banco MEX	✓ código banco PER	catBanco.IdCatPais ya distingue MEX / PER
IdCatMoneda → catMoneda.ClaveMoneda	✓ MXN / USD	⚠ PEN no existe en catálogo	Falta registro PEN — cambio de datos, no de esquema
NumeroTarjeta varchar(20)	—	—	Sin uso para transferencias interbancarias

2. Cambios R16 planeados — impacto sobre CCI

Cambio R16	Impacto MEX	Impacto PER
IdRegion en EmpresaDatosBancarios	✓ filtra cuentas MEX	✓ clave — indica que el campo Clabe contiene un CCI cuando RegionClave = 'PER'
vEmpresaDatosBancarios (vista nueva)	✓ expone RegionClave, ClaveBanco, Clabe	✓ misma vista sirve — RegionClave = 'PER' señala semántica CCI

3. Punto crítico: nombre del campo clabe

El campo se llama clabe pero para PER almacenaría el CCI. Dos opciones evaluadas:

Opción	Cambio requerido	Impacto
A — Reutilizar `Clabe` as-is (seleccionada)	Ninguno en esquema	IdRegion provee el contexto suficiente para saber si el valor es CLABE o CCI. Sin ALTER TABLE adicional, sin

		riesgo en EDMX ni en BO existentes.
B — Renombrar a `ClaveInterbancaria`	sp_rename + EDMX + NuGet + todos los BO/Controllers	Semánticamente correcto pero rompe referencias existentes; riesgo de regresión alto en R16. Diferir a refactor futuro.

Decisión: Opción A. IdRegion ya da el contexto. El campo aguanta ambos formatos. Renombrar en R16 agrega riesgo sin ganancia funcional.

4. Requisitos pendientes para habilitar CCI (Perú — fuera de alcance R16)

Requisito	Tipo de cambio	Objeto
Agregar moneda PEN	INSERT en catálogo	catMoneda
Agregar bancos PE (BCP, BBVA Perú, Interbank, etc.)	INSERT en catálogo	catBanco CON IdCatPais = PER
Cuentas GOLPERU	INSERT operacional	DatosBancarios + EmpresaDatosBancarios con IdRegion = PER

Ninguno de estos cambios requiere ALTER TABLE. La estructura actual de DatosBancarios soporta CCI sin modificaciones de esquema.

5. Conclusión

La estructura actual de DatosBancarios **soporta CCI sin cambios de esquema**. Los cambios R16 planeados (IdRegion + vista vEmpresaDatosBancarios) son suficientes para regionalizar y distinguir CLABE/CCI en tiempo de consulta. Solo faltan **datos** (PEN en catMoneda, bancos PE en catBanco) — no estructura ni endpoints adicionales.

Impacto técnico

1. Orden de ejecución y archivos

#	Paso	Archivo / Objeto	Tipo
1	Script BD Paso 1	Scripts\R16\FU-001_EmpresaDatosBancarios_IdRegion.sql	Nuevo
2	Update Model EDMX	Core.Pqf\ProquifaDotNetContext\ProquifaDotNetModel.edmx	Modificar

3	Script BD Paso 2	Scripts\R16\FU-001_vEmpresaDatosBancarios.sql	Nuevo
4	Update Model EDMX (2ª vez)	Core.Pqf\ProquifaDotNetContext\ProquifaDotNetModel.edmx	Modificar
5	Generar NuGet Core.Pqf	—	Build
6	Actualizar NuGet en ProquifaDotNet-R17	—	Update
7	Corrección ObtenerTodos()	Logic.Pqf.Catalogos\...\EmpresaDatosBancariosB0.Extensions.cs	Modificar
8	Nuevo BO	Logic.Pqf.Catalogos\...\vEmpresaDatosBancariosB0.cs	Nuevo
9	Nuevo Controller	WebApi.Catalogos\...\vEmpresaDatosBancariosController.cs	Nuevo

2. Archivos sin cambio en R16

Archivo	Motivo
EmpresaDatosBancariosController.cs (PUT/DELETE)	Solo para Soporte a la Producción — sin cambios
EmpresaDatosBancariosDetalleB0.cs	La vista reemplaza su uso en módulos consumidores
DatosBancariosB0.Empresa.Extensions.cs	Ya filtra cuentas vigentes correctamente

Manejo de errores y excepciones

Escenario	Comportamiento esperado
Usuario sin IdRegion en token	TryExecute captura excepción → 500 InternalServerError con log
Vista retorna lista vacía	204 NoContent — no es error; región válida sin cuentas vigentes



ObtenerPorId sin resultado	204 NoContent — cuenta no existe o está inactiva
BD no disponible	TryExecute captura → 500 InternalServerError; sin exponer stack trace
EDMX desincronizado con BD	Error de compilación en Core.Pqf — se detecta antes de generar NuGet

Estrategia de pruebas

1. Pruebas funcionales (Criterios de Aceptación en DEV)

- POST /vEmpresaDatosBancarios con usuario región MEX retorna solo cuentas GOL/MUN/PRO/PQF vigentes.
- Una cuenta con Activo = 0 no aparece en la respuesta.
- ObtenerTodos() corregido no retorna cuentas con Activo = false.
- GET /vEmpresaDatosBancarios?idEmpresaDatosBancarios={guid} retorna 204 si cuenta inactiva.

2. Pruebas técnicas

a. Unitarias

- vEmpresaDatosBancariosB0.Lista: filtro Activo = true e IdRegion siempre aplicados.
- vEmpresaDatosBancariosB0.ObtenerPorId: retorna null si cuenta Activo = false.
- ObtenerTodos() corregido: no retorna registros con Activo = false.

b. Integración

- Seed: 3 cuentas Activo=1 región MEX + 2 cuentas Activo=0 + 1 cuenta región PER.
- POST /vEmpresaDatosBancarios con usuario MEX → exactamente 3 registros.
- Ningún registro retornado pertenece a región PER ni tiene Activo=0.

3. Casos críticos

Caso	Verificación esperada
EDMX no actualizado tras ALTER TABLE	Error de compilación en Core.Pqf — bloquea el build
NuGet desactualizado en ProquifaDotNet-R17	Entidad sin IdRegion → error en tiempo de ejecución
Dos módulos consultando simultáneamente	Sin interferencia — consultas de solo lectura sobre la vista



Usuario sin región asignada	500 con log — no puede continuar sin IdRegion
-----------------------------	---

Control de versiones

Versión	Fecha	Autor	Tipo de Cambio	Descripción del cambio	Aprobó
1.0	19 jun 2026	Jose Albe...	Creación	Creación del documento.	

